

COGNIZANT DIGITAL NURTURE 5.0

Week 1 – Design Patterns & Data Structures and Algorithms

Name: Gopiseti Gowtham

Exercise 1: Singleton Pattern

Objective

To implement the Singleton Design Pattern so that only one instance of the Logger class can be created and shared throughout the application.

Source Code

Logger.java

```
package DesignPatterns.Exercise1_SingletonPattern;

public class Logger {

    private static Logger instance;

    private Logger() {
        System.out.println("Logger instance created.");
    }
}
```

```
public static Logger getInstance() {  
  
    if (instance == null) {  
  
        instance = new Logger();  
  
    }  
  
    return instance;  
  
}  
  
public void log(String message) {  
  
    System.out.println("LOG: " + message);  
  
}  
  
}
```

SingletonTest.java

```
package DesignPatterns.Exercise1_SingletonPattern;  
  
public class SingletonTest {  
  
    public static void main(String[] args) {  
  
        Logger logger1 = Logger.getInstance();  
        logger1.log("Application started.");  
  
        Logger logger2 = Logger.getInstance();  
        logger2.log("Application is running.");  
  
    }  
  
}
```

```

        if (logger1 == logger2) {

            System.out.println("Only one Logger instance exists.");

        } else {

            System.out.println("Multiple Logger instances exist.");

        }

    }

}

```

Output

The screenshot shows an IDE with the following components:

- Project View:** A tree structure on the left showing the project hierarchy. The 'SingletonTest' file is selected under the 'DesignPatterns' folder.
- Editor:** The main window displays the code for 'SingletonTest.java'. The code is as follows:


```

1 package DesignPatterns.Exercise1_SingletonPattern;
2
3 public class SingletonTest {
4     // Gopiseti Gowtham
5
6     public static void main(String[] args) {
7         // Gopiseti Gowtham
8
9         Logger logger1 = Logger.getInstance();
10        logger1.log("Application started.");
11
12        Logger logger2 = Logger.getInstance();
13        logger2.log("Application is running.");
14
15        if (logger1 == logger2) {
16            System.out.println("Only one Logger instance exists.");
17        } else {
18            System.out.println("Multiple Logger instances exist.");
19        }
20    }
21 }
      
```
- Run View:** The bottom panel shows the output of the program:


```

/Library/Java/JavaVirtualMachines/openjdk-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=51780 -Dfile.encoding=UTF-8 -cL
Logger instance created.
LOG: Application started.
LOG: Application is running.
Only one Logger instance exists.

Process finished with exit code 0
      
```
- IDE Project Settings:** A notification at the bottom right suggests adding IDE project settings to Git, with options to 'View Files', 'Always Add', or 'Don't Ask Again'.

Exercise 2: Factory Method Pattern

Objective

To implement the Factory Method Design Pattern for creating different document objects without exposing the object creation logic to the client.

Source Code

FactoryTest.java

```
package DesignPatterns.Exercise2_FactoryMethodPattern;

public class FactoryTest {

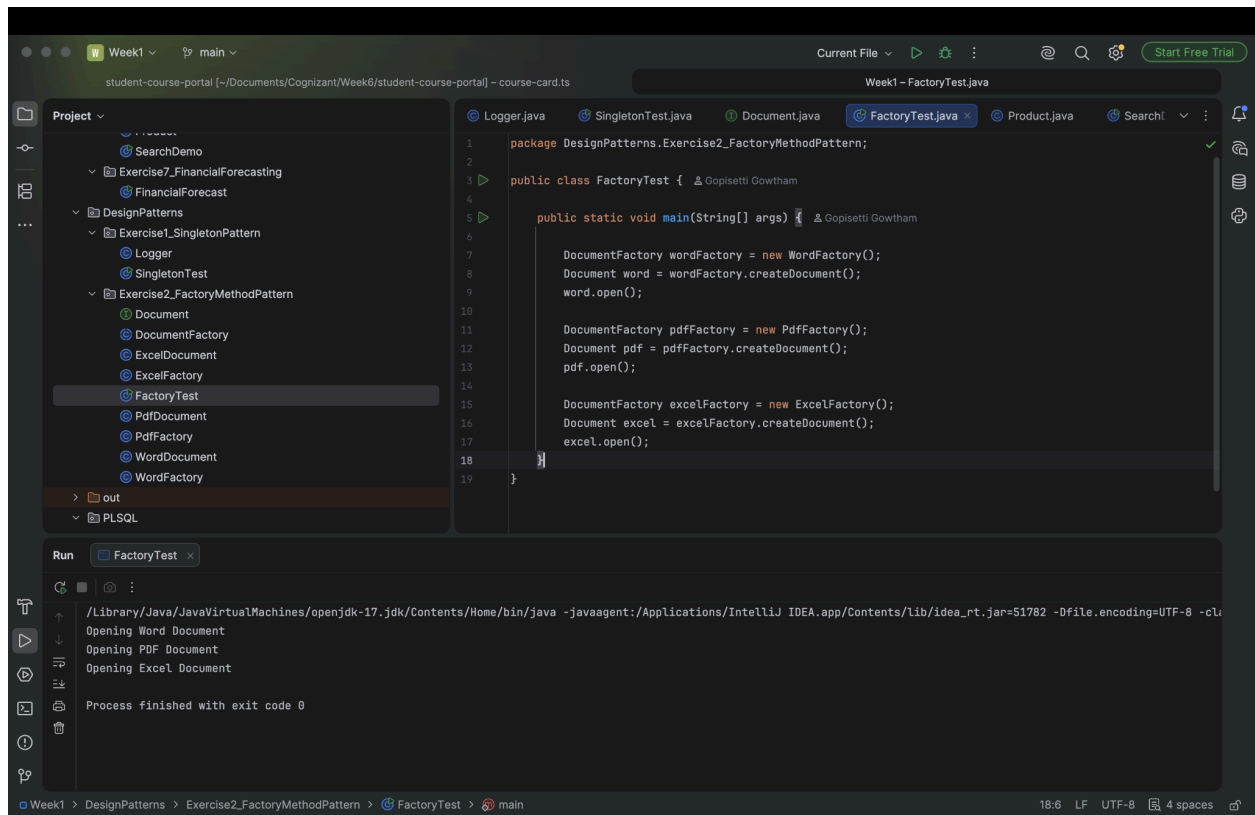
    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}
```

Output



The screenshot shows an IDE window titled "Week1 - FactoryTest.java". The left sidebar displays a project tree with the following structure:

- Project
 - SearchDemo
 - Exercise7_FinancialForecasting
 - FinancialForecast
 - DesignPatterns
 - Exercise1_SingletonPattern
 - Logger
 - SingletonTest
 - Exercise2_FactoryMethodPattern
 - Document
 - DocumentFactory
 - ExcelDocument
 - ExcelFactory
 - FactoryTest
 - PdfDocument
 - PdfFactory
 - WordDocument
 - WordFactory
 - out
 - PLSQL

The main editor displays the code for `FactoryTest.java`:

```
1 package DesignPatterns.Exercise2_FactoryMethodPattern;
2
3 public class FactoryTest {
4     // Gopiseti Gowtham
5
6     public static void main(String[] args) {
7         // Gopiseti Gowtham
8
9         DocumentFactory wordFactory = new WordFactory();
10        Document word = wordFactory.createDocument();
11        word.open();
12
13        DocumentFactory pdfFactory = new PdfFactory();
14        Document pdf = pdfFactory.createDocument();
15        pdf.open();
16
17        DocumentFactory excelFactory = new ExcelFactory();
18        Document excel = excelFactory.createDocument();
19        excel.open();
20    }
21 }
```

The bottom panel shows the Run configuration for `FactoryTest` and the execution output:

```
Run FactoryTest
/Library/Java/JavaVirtualMachines/openjdk-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=51782 -Dfile.encoding=UTF-8 -cli
Opening Word Document
Opening PDF Document
Opening Excel Document
Process finished with exit code 0
```

Exercise 3: E-Commerce Search

Objective

To implement Linear Search and Binary Search algorithms for searching products in an e-commerce product list and compare their functionality.

Source Code

Product.java

```
package DataStructuresAndAlgorithms.Exercise2_EcommerceSearch;
```

```
public class Product {

    int productId;

    String productName;

    String category;

    public Product(int productId, String productName, String category) {

        this.productId = productId;

        this.productName = productName;

        this.category = category;

    }

}
```

SearchDemo.java

```
package DataStructuresAndAlgorithms.Exercise2_EcommerceSearch;

public class SearchDemo {

    public static int linearSearch(Product[] products, String key) {

        for (int i = 0; i < products.length; i++) {

            if (products[i].productName.equalsIgnoreCase(key)) {

                return i;

            }

        }

        return -1;

    }

}
```

```
}

public static int binarySearch(Product[] products, String key) {

    int low = 0;

    int high = products.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        int result = products[mid].productName.compareToIgnoreCase(key);

        if (result == 0) {

            return mid;

        } else if (result < 0) {

            low = mid + 1;

        } else {

            high = mid - 1;

        }

    }

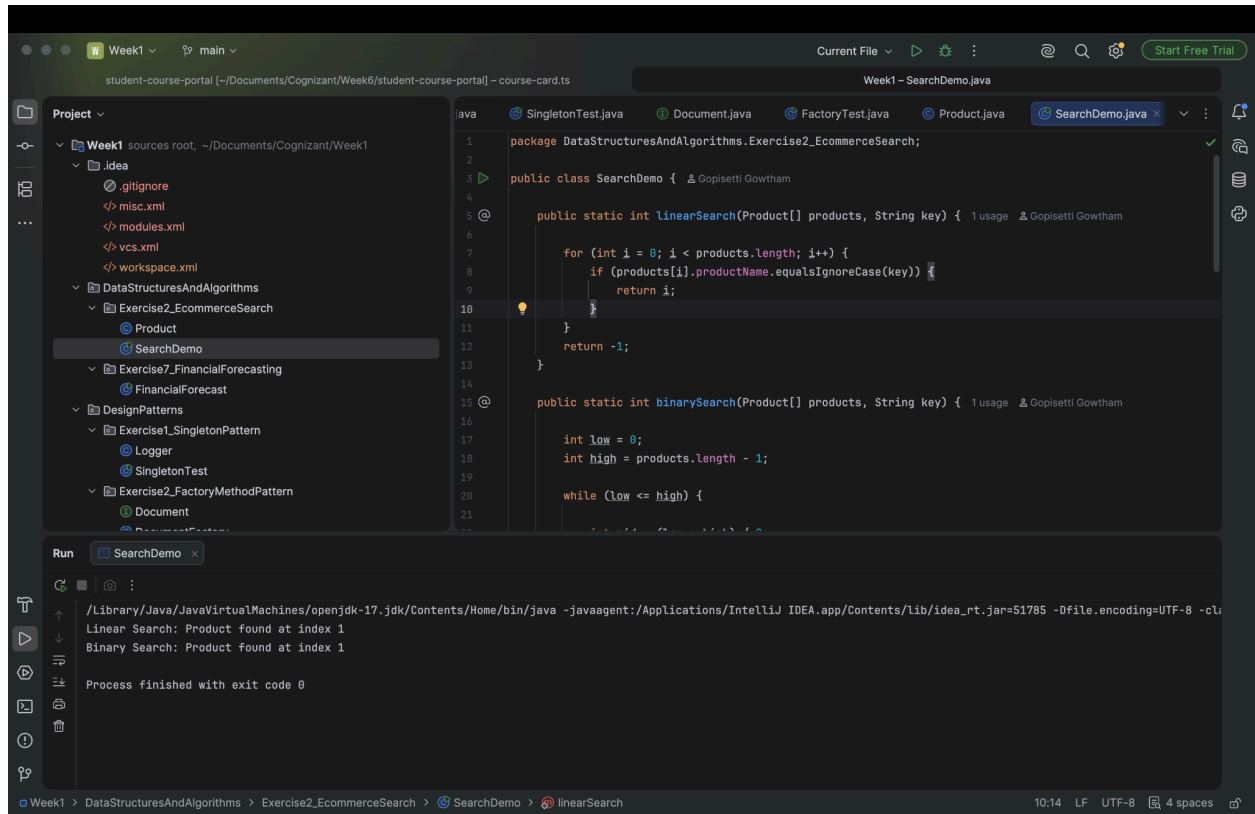
    return -1;

}

public static void main(String[] args) {
```

```
Product[] products = {  
    new Product(101, "Laptop", "Electronics"),  
    new Product(102, "Mobile", "Electronics"),  
    new Product(103, "Tablet", "Electronics")  
};  
  
int linearResult = linearSearch(products, "Mobile");  
  
if (linearResult != -1) {  
    System.out.println("Linear Search: Product found at index " +  
linearResult);  
} else {  
    System.out.println("Linear Search: Product not found");  
}  
  
int binaryResult = binarySearch(products, "Mobile");  
  
if (binaryResult != -1) {  
    System.out.println("Binary Search: Product found at index " +  
binaryResult);  
} else {  
    System.out.println("Binary Search: Product not found");  
}  
}
```

Output



The screenshot shows an IDE with a project named 'Week1'. The 'Project' view on the left shows the file structure, including 'DataStructuresAndAlgorithms' and 'Exercise2_EcommerceSearch'. The 'SearchDemo.java' file is open in the editor, showing two methods: 'linearSearch' and 'binarySearch'. The 'Run' view at the bottom shows the output of the program, which is a command line execution of 'Linear Search' and 'Binary Search', both finding a product at index 1. The process finished with exit code 0.

```
package DataStructuresAndAlgorithms.Exercise2_EcommerceSearch;

public class SearchDemo {

    public static int linearSearch(Product[] products, String key) {
        for (int i = 0; i < products.length; i++) {
            if (products[i].productName.equalsIgnoreCase(key)) {
                return i;
            }
        }
        return -1;
    }

    public static int binarySearch(Product[] products, String key) {
        int low = 0;
        int high = products.length - 1;

        while (low <= high) {
```

```
/Library/Java/JavaVirtualMachines/openjdk-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=51785 -Dfile.encoding=UTF-8 -cl
Linear Search: Product found at index 1
Binary Search: Product found at index 1

Process finished with exit code 0
```

Exercise 4: Financial Forecasting

Objective

To implement a recursive algorithm for financial forecasting to predict the future value of an investment based on the current value, annual growth rate, and number of years.

Source Code

FinancialForecast.java

[illegible]

```

        years);

    System.out.printf("Future Value after %d years = %.2f%n",
        years,
        futureValue);
}
}

```

Output

The screenshot displays the IntelliJ IDEA IDE interface. The left sidebar shows the project structure with the following hierarchy:

- Week1
 - .idea
 - .gitignore
 - misc.xml
 - modules.xml
 - vcs.xml
 - workspace.xml
 - DataStructuresAndAlgorithms
 - Exercise2_EcommerceSearch
 - Product
 - SearchDemo
 - Exercise7_FinancialForecasting
 - FinancialForecast (selected)
 - DesignPatterns
 - Exercise1_SingletonPattern
 - Logger
 - SingletonTest
 - Exercise2_FactoryMethodPattern

The main editor window shows the `FinancialForecast.java` file with the following code:

```

package DataStructuresAndAlgorithms.Exercise7_FinancialForecasting;

public class FinancialForecast {
    @Gopiseti Gowtham

    public static double predictFutureValue(double currentValue, 2 usages @Gopiseti Gowtham
        double growthRate,
        int years) {

        if (years == 0) {
            return currentValue;
        }

        return predictFutureValue(
            currentValue * (1 + growthRate),
            growthRate,
            years - 1);
    }

    public static void main(String[] args) {
        @Gopiseti Gowtham

        double currentValue = 10000;
        double growthRate = 0.10;
        int years = 5;

        predictFutureValue(currentValue, growthRate, years);
    }
}

```

The Run window at the bottom shows the execution output:

```

/Library/Java/JavaVirtualMachines/openjdk-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=51787 -Dfile.encoding=UTF-8 -cl
Future Value after 5 years = 16105.10

Process finished with exit code 0

```

The status bar at the bottom indicates the file is at line 18, column 1, using UTF-8 encoding with 4 spaces.